Equation to Animation

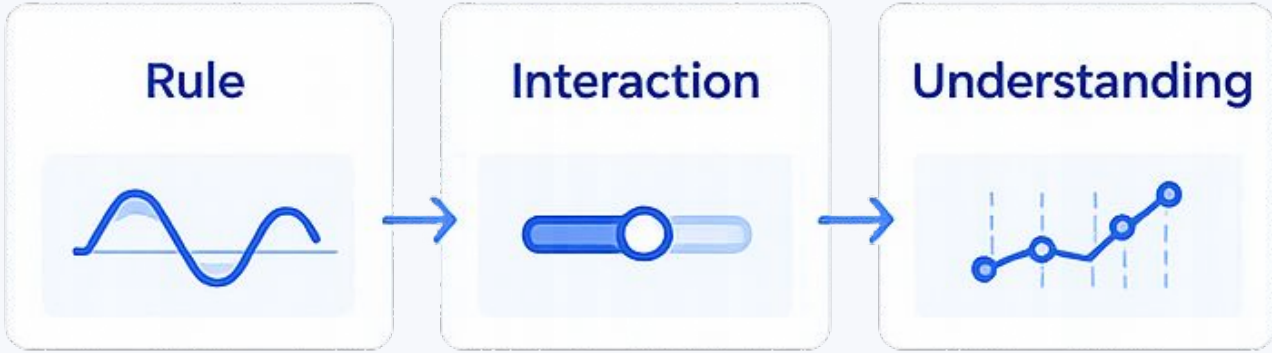
Crafting Dynamic Math Visuals on the Web

Courtney Yatteau

From formula to pixels to motion to insight

Why math visuals work on the web

- Small rules become visible behavior
- Input creates immediate feedback
- Patterns become easier to understand



OUTCOMES

Practical promise

A reusable architecture

Inputs ⇒ rule ⇒ state ⇒ render ⇒ explain

A library decision model

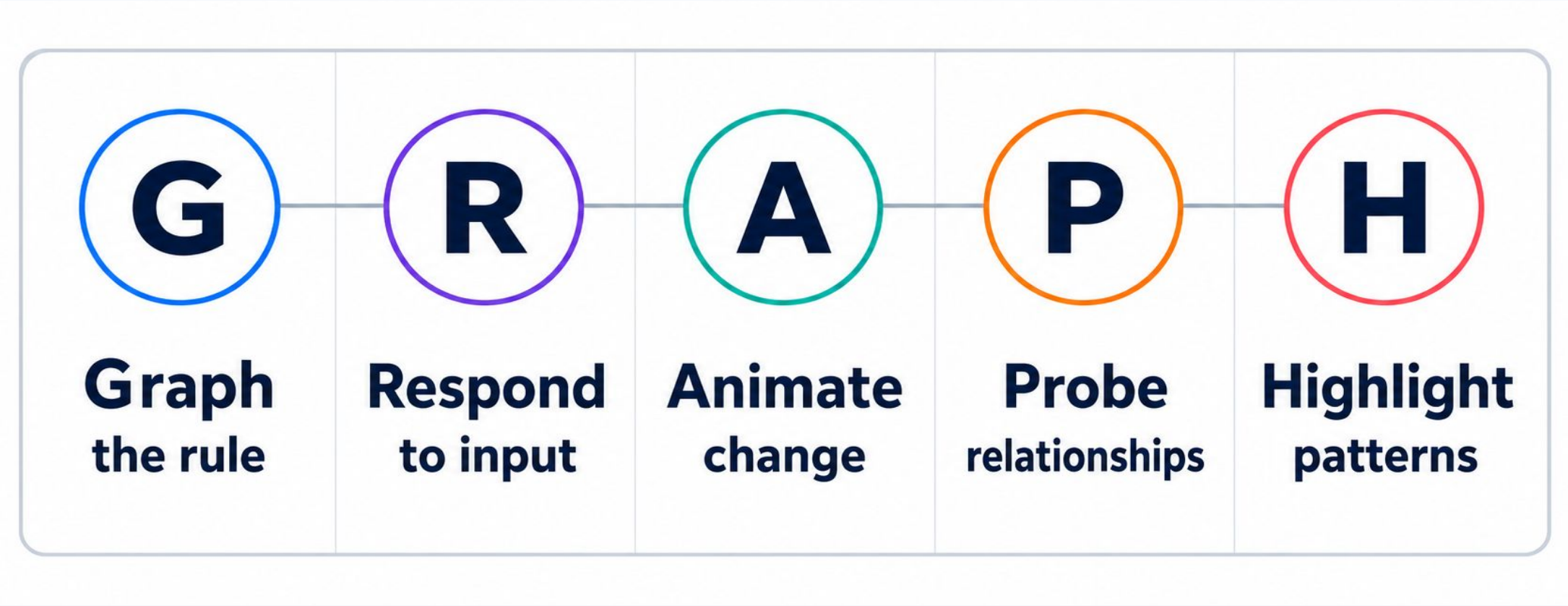
Choose the right level of abstraction

Six remixable demos

Turn each demo into a lab, tutorial, or content idea

THROUGHLINE

GRAPH: one lens for every demo



STACK

Use the highest-level tool that still lets the math lead

Config-first

Chart.js

Fast chart setup

Plot

Quick statistical views

More direct control

SVG + WAAPI

Transforms become motion

D3.js

Scales and transitions

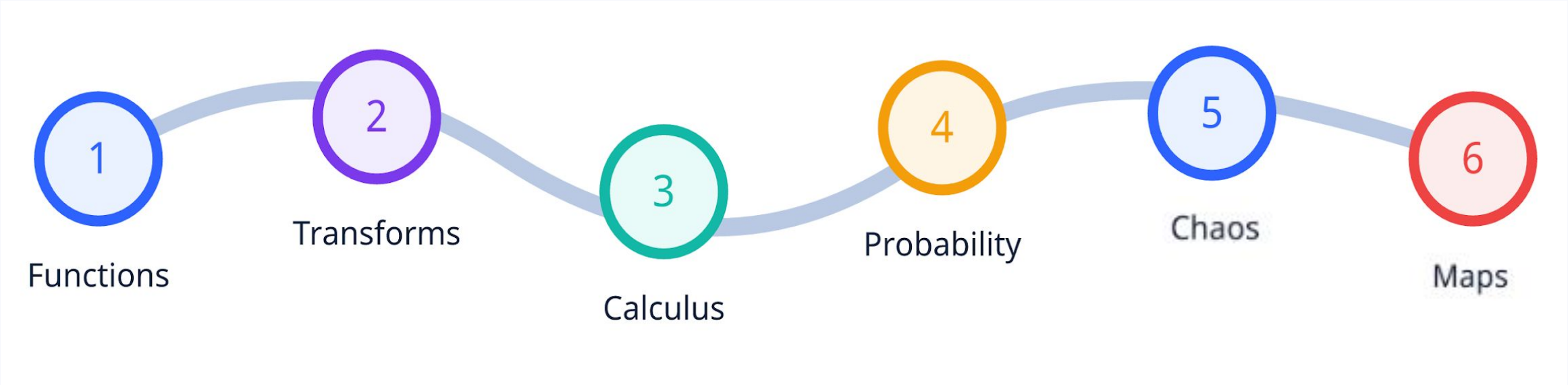
Leaflet

Distance on a real map

Animation loop

requestAnimationFrame

Six demos, one climb in complexity



familiar ⇒ interactive ⇒ analytical ⇒ emergent ⇒ spatial

Demo 1

Function Playground

Changing parameters changes behavior.

Style can react to data

```
segment: {
  borderColor(ctx) {
    const delta = ctx.p1.parsed.y - ctx.p0.parsed.y;
    return Math.abs(delta) < 0.02 ? teal : delta > 0 ? blue : orange;
  }
}
```

GRAPH: one lens for every demo

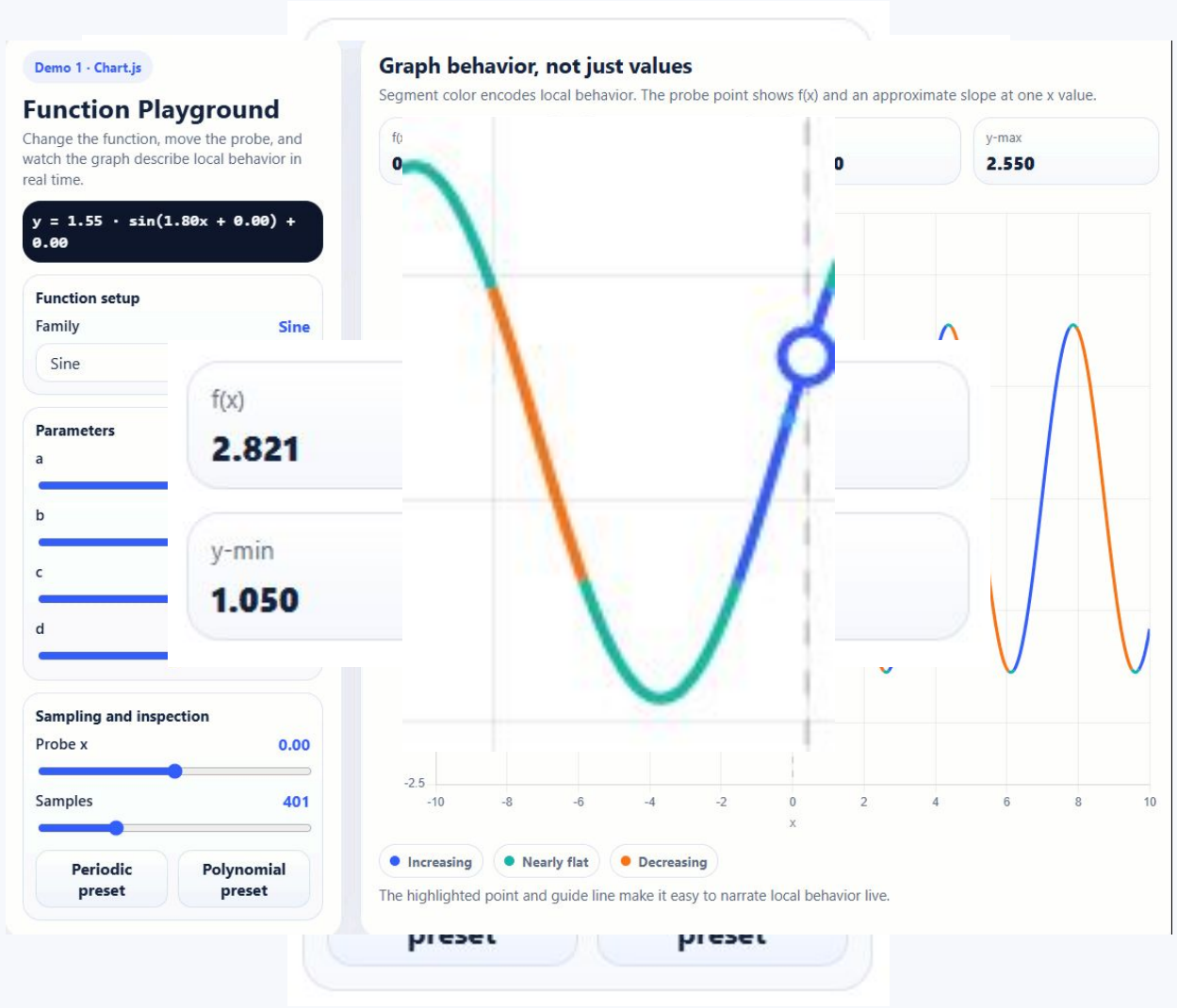
G **Graph** the rule

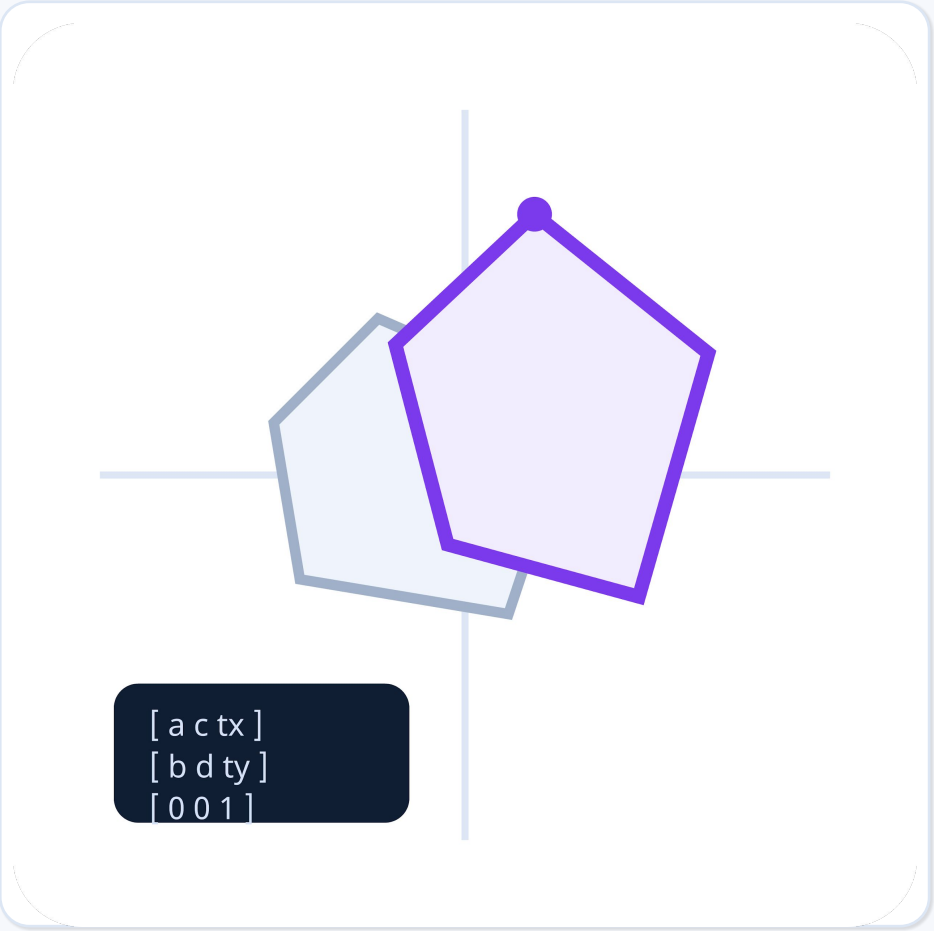
R **Respond** to input

A **Animate** change

P **Probe** relationships

H **Highlight** patterns





Demo 2

Transformation Playground

Matrix changes become visible motion.

Transforms compose into one matrix

```
const transformString = `matrix(${a}, ${b}, ${c}, ${d}, ${e}, ${f})`;

shape.animate(
  [
    { transform: previousTransform },
    { transform: transformString }
  ],
  { duration: 650, easing: “cubic-bezier(.2,.8,.2,1)”, fill: “forwards” }
);

shapeEl.style.transform = transformString;
```

GRAPH: one lens for every demo

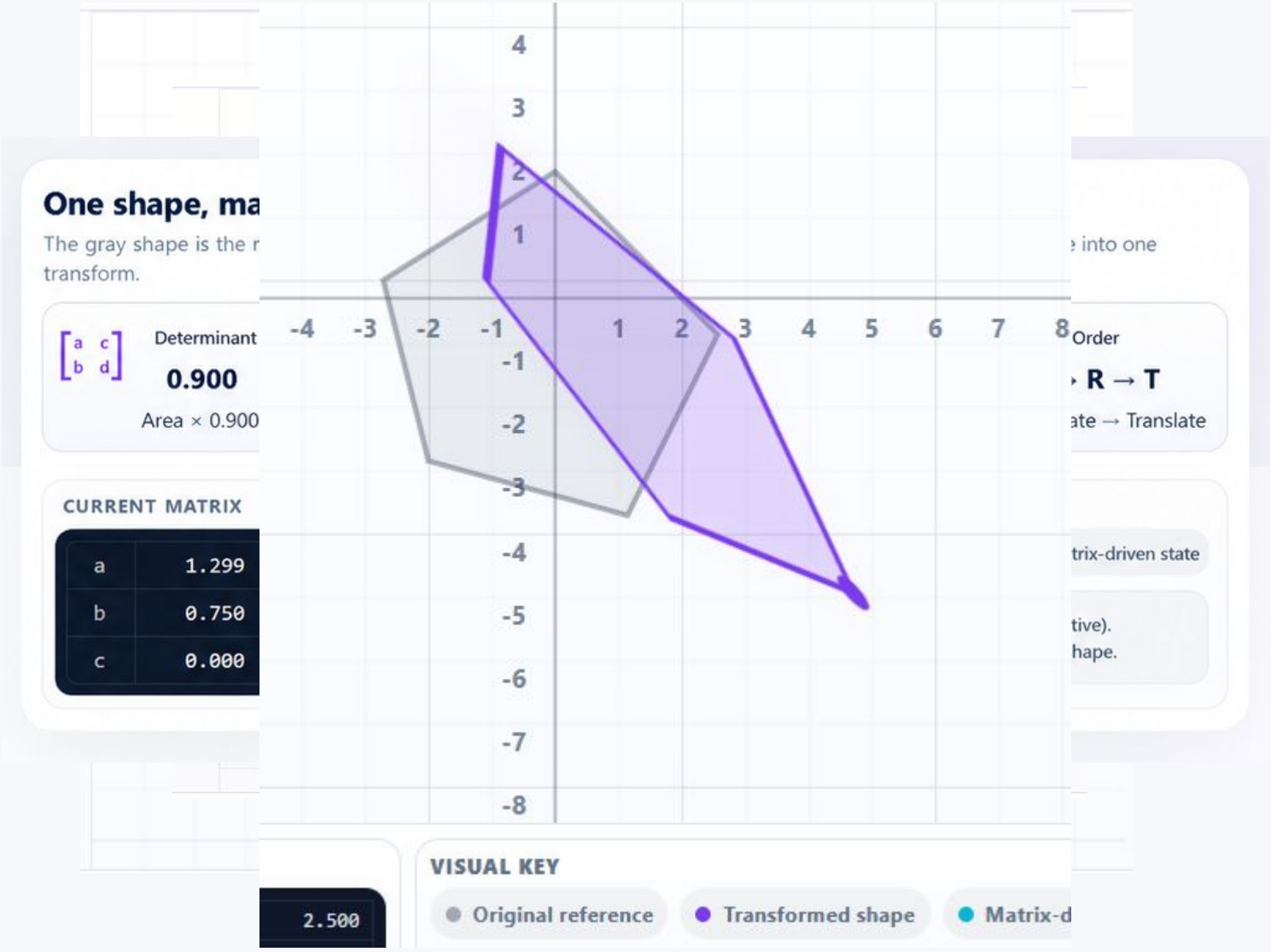
G **Graph**
the rule

R **Respond**
to input

A **Animate**
change

P **Probe**
relationships

H **Highlight**
patterns



Demo 3

Calculus Motion Lab

Limits and accumulation as motion.

D3 maps math values to motion

```
const x = d3.scaleLinear()
  .domain([-6, 6])
  .range([margin.left, width - margin.right]);

const y = d3.scaleLinear()
  .domain([-8, 8])
  .range([height - margin.bottom, margin.top]);

point0.transition(transition)
  .attr('cx', x(x0))
  .attr('cy', y(y0));
```

GRAPH: one lens for every demo

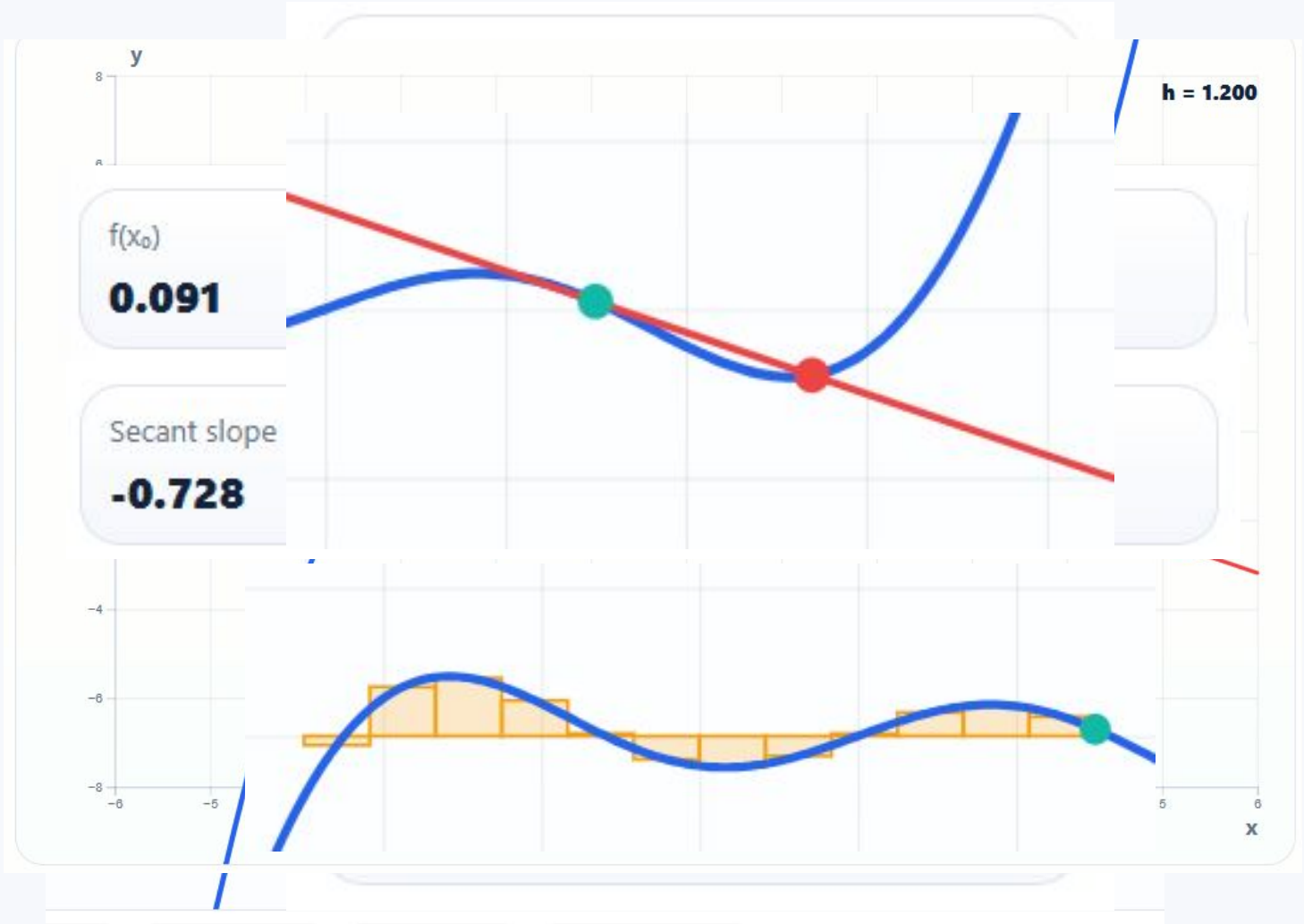
G **Graph**
the rule

R **Respond**
to input

A **Animate**
change

P **Probe**
relationships

H **Highlight**
patterns



Demo 4

Observable Plot

Probability and Statistics Explorer

Binning turns repeated values into a shape

```
Plot.plot({
  marks: [
    Plot.rectY(values,
      Plot.binX({ y: 'count' }, {
        x: d => d,
        thresholds: 26,
        fill: 'rgba(14,165,233,0.55)'
      })
    )
  ]
});
```

GRAPH: one lens for every demo

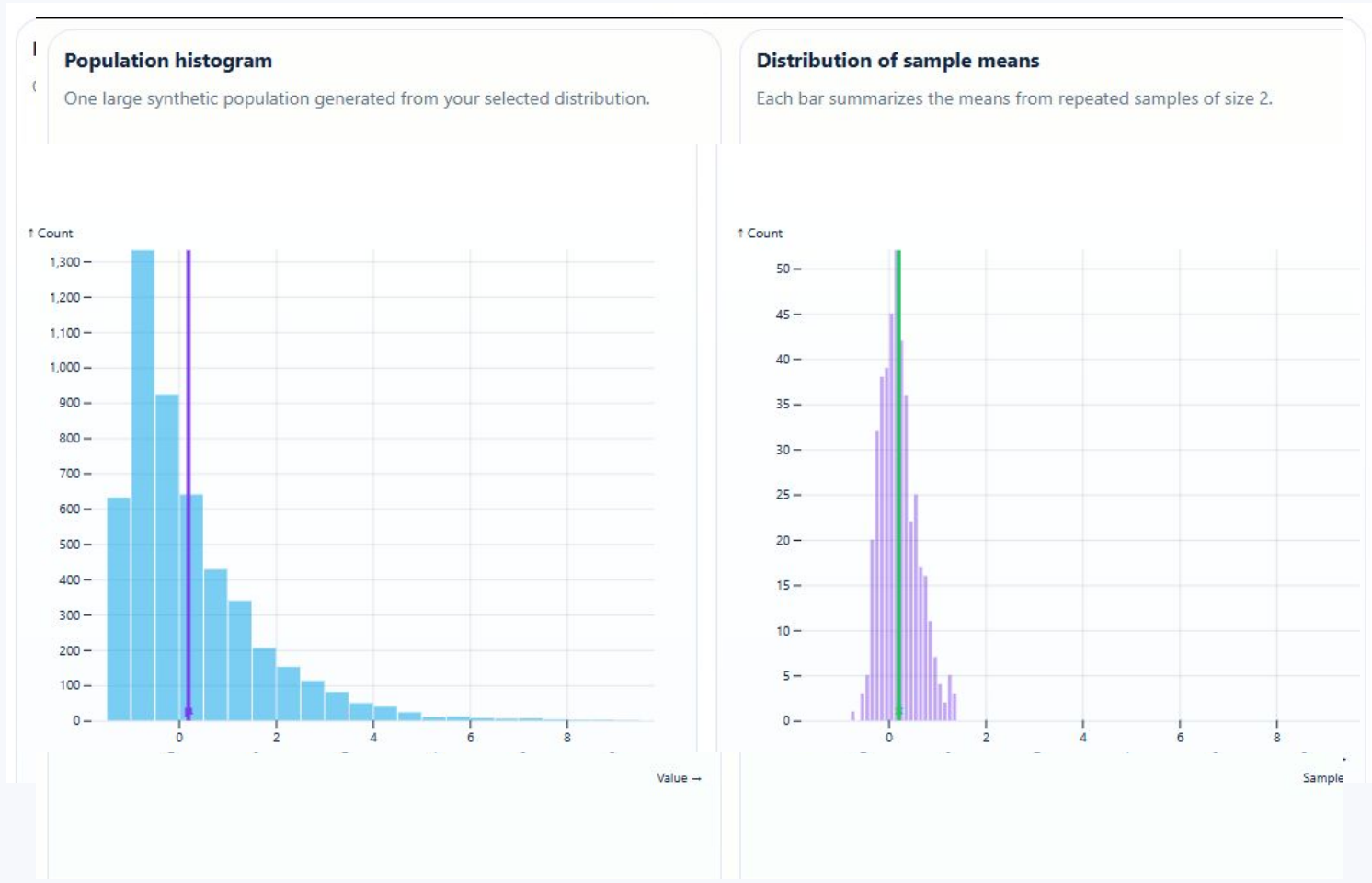
G **Graph** the rule

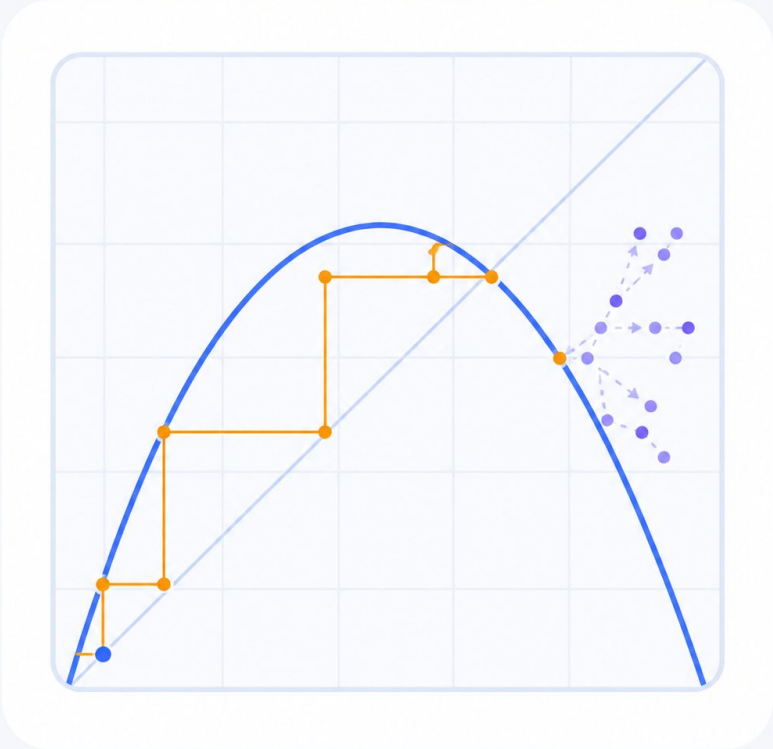
R **Respond** to input

A **Animate** change

P **Probe** relationships

H **Highlight** patterns





Demo 5

Chaos Explorer

One tiny rule becomes surprising behavior.

Iteration creates the visual complexity

```
function iterate(r, seed, n) {
  let x = seed;
  const values = [];

  for (let i = 0; i < n; i++) {
    x = r * x * (1 - x);
    values.push(x);
  }

  return values;
}
```

GRAPH: one lens for every demo

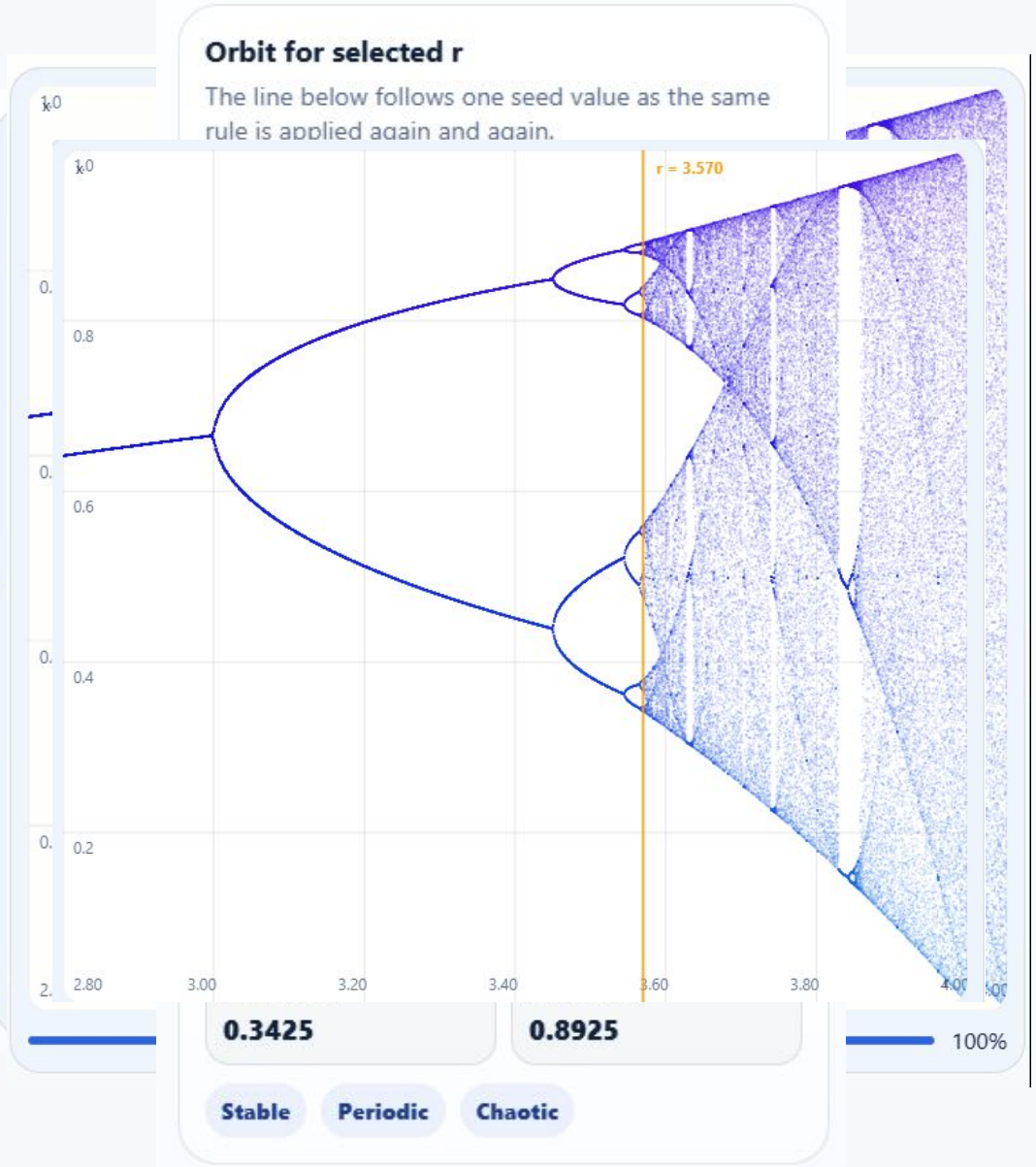
G **Graph**
the rule

R **Respond**
to input

A **Animate**
change

P **Probe**
relationships

H **Highlight**
patterns



Demo 6

Map Radius Explorer

Distance becomes reachability.

A distance threshold drives the interface

```
record.distance = from.distanceTo(record.latlng);
const isActive = currentRadiusMeters >= record.distance;
rippleCircle.setRadius(currentRadiusMeters);
requestAnimationFrame(step);
```

GRAPH: one lens for every demo

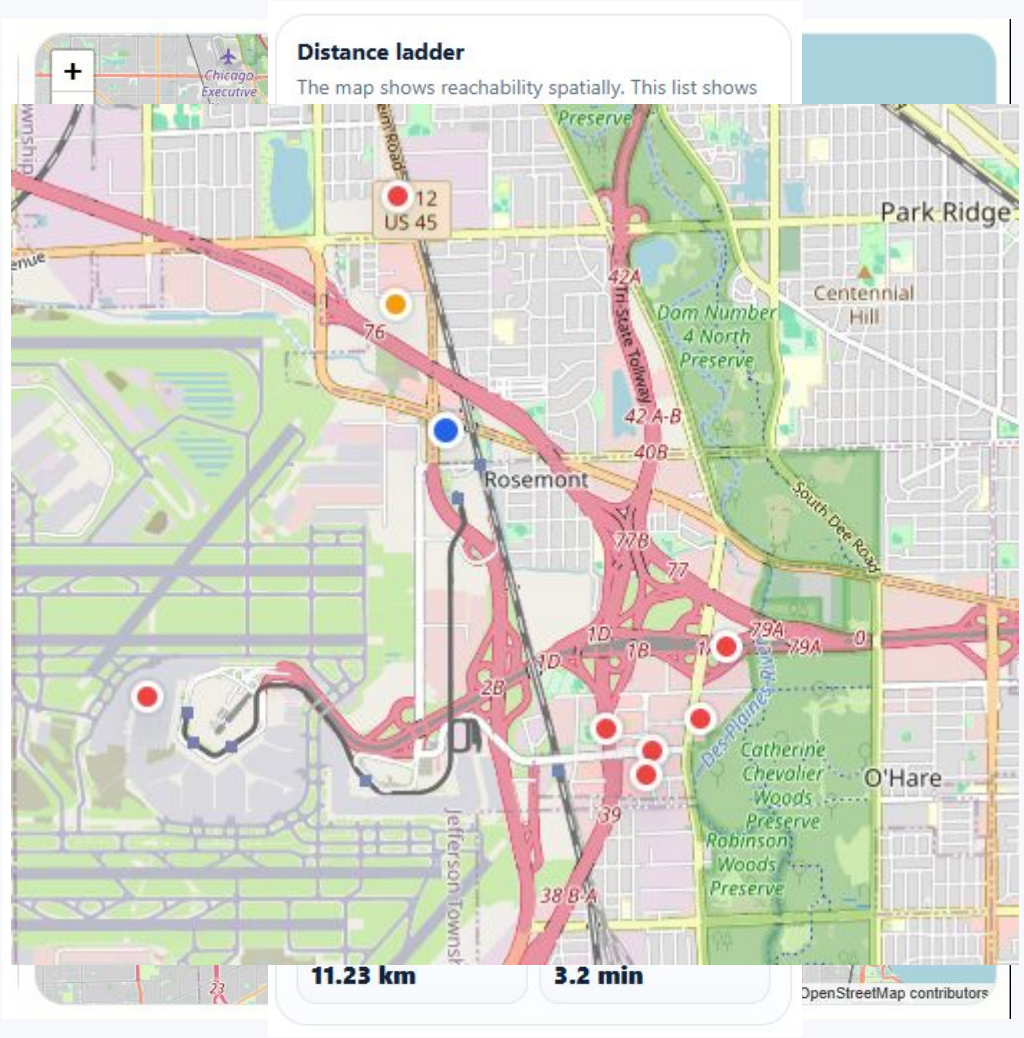
G **Graph**
the rule

R **Respond**
to input

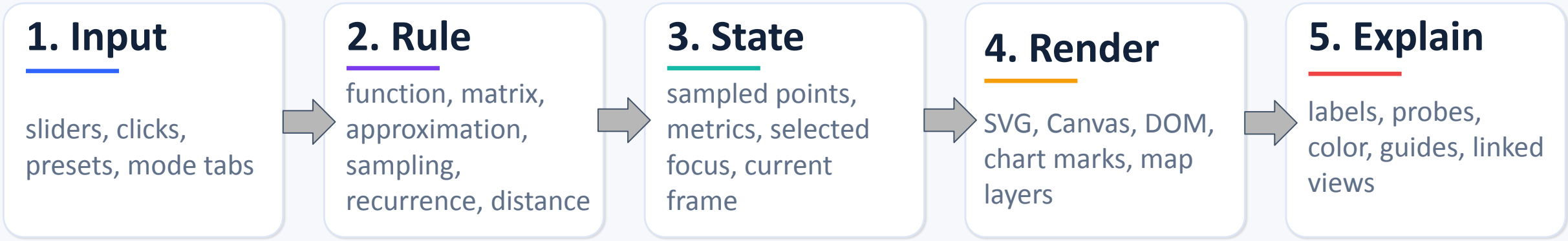
A **Animate**
change

P **Probe**
relationships

H **Highlight**
patterns



What repeats across all demos



The math changes. The interaction architecture repeats.

TAKEAWAY

A simple recipe for building your own math visual

Start with one rule

Make it easy to say out loud.

Make it visible

Choose the smallest visual that reveals the rule.

Add one interaction

One control. One animation. One annotation.

Choose the tool last

Use the lowest abstraction only when the idea needs it.

Thank you, JS Tek!

Courtney Yatteau

@c_yatteau

courtneyyatteau

@cyatteau.bsky.social